

Timing Investigation Plan

Status: queued for the next session (post-2026-05-20).

Trigger: ~25 ms wall-clock jitter floor observed on René at 16 stepsPerCycle, independent of BPM. See [\[\[project_timing_jitter_investigation_queued\]\]](#).

Goal

Map every component's contribution to end-to-end timing latency and jitter, identify the source of the observed 25 ms BEAM-side floor, and produce a calibrated picture of the rig as a hard-real-time system.

This is the load-bearing substrate for everything else in the music- making stack. We treat it as primary infrastructure — investigate deeply, fix at root. See [\[\[feedback_dont_patch_foundational_substrate\]\]](#).

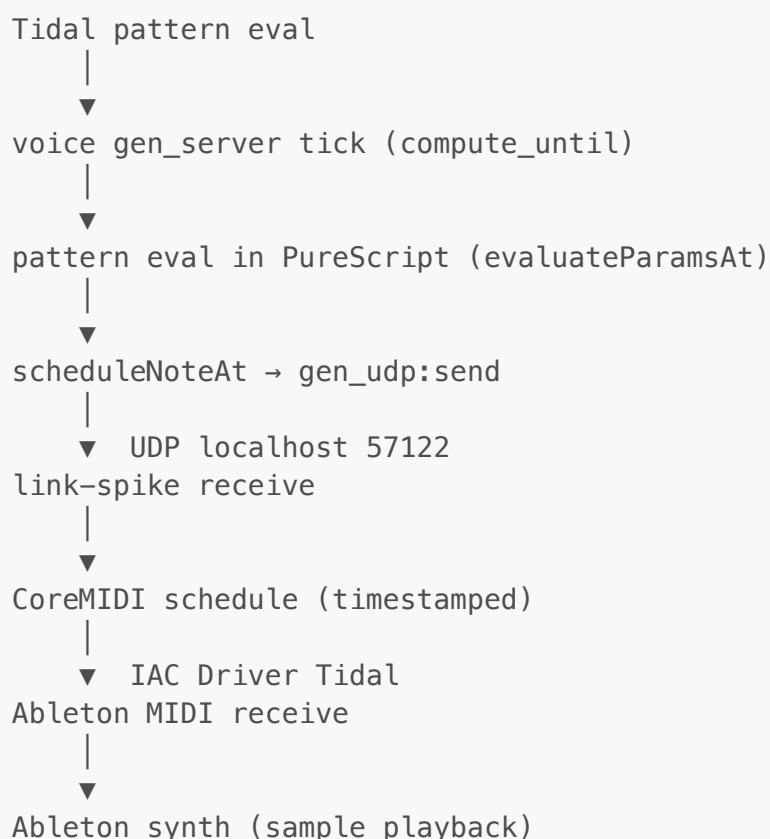
Principle

Measure each stage in isolation, then compose. The hypothesis is that one or two stages dominate the variance and the rest are negligible. Identifying WHICH is the first investigation — we don't optimise anything until we know what to optimise.

Each phase isolates a delta from the previous phase, so the source of any new variance is unambiguous.

Signal chain to instrument

Each arrow is a potential latency / jitter contributor:





What we measure per stage

For each phase:

- **Latency** — constant offset between scheduled time and observed time
- **Jitter** — standard deviation of that offset across many events
- **Drift** — does the offset shift slowly over minutes
- **Worst case** — max excursion (GC pauses, scheduler pre-emption)
- **Percentiles** — p50 / p95 / p99 (mean is misleading for long-tail distributions)

Phases

The phases run in order. Each one tells us a delta from the previous. Stop early if a phase is clean — no need to keep instrumenting beyond the bottleneck.

Phase 0 — Tool sanity / analysis floor

What: Record Ableton's internal metronome at 60 BPM for 60 seconds. Run our onset-detection pipeline on the audio.

Why: Establish the floor of our analysis tools. If we see >1 ms jitter on a metronome (which should be sample-accurate inside Live), that's tool noise we need to subtract from everything else.

Acceptance: Detected onsets within <1 ms of the bar grid. Confirms the analysis pipeline. If not, fix the pipeline before proceeding.

Phase 1 — Tidal → MIDI only, no audio

What: Sparse Tidal pattern — a single C4 note at every Tidal cycle, slow tempo (60 BPM). Emit MIDI via IAC to Ableton. A Max for Live patch in Live captures MIDI-receive timestamps relative to Live's transport.

Why: Bypasses audio buffers entirely. Isolates BEAM + link-spike + CoreMIDI + Ableton MIDI-input variance. No DAW synth, no audio I/O.

Acceptance: <5 ms jitter expected. Numbers above that indicate a problem in the MIDI path before any audio enters the picture.

Phase 2 — Tidal → MIDI → audio

What: Same sparse Tidal pattern. Ableton plays a sharp transient (short drum sample). Record Ableton's master out at 96 kHz / 24 bit. Onset-detect the recording.

Why: Adds Ableton's synth and audio buffer to the chain. Phase 2 minus Phase 1 isolates Ableton's MIDI-to-audio contribution.

Acceptance: Phase 1 jitter + ~5–15 ms constant offset (Ableton buffer). The offset is a constant — calibrate once, subtract.

Phase 3 — Same chain, dense pattern

What: Replace the sparse pattern with a dense one — 16 notes per cycle at the same slow tempo. All the same chain (Tidal pattern, no vmod, no cross-machine).

Why: This is where I expect the 25 ms BEAM-side floor to first appear. If Phase 3 is clean while René at 16 stepsPerCycle isn't, the problem is vmod-specific and we move to Phase 4. If Phase 3 already shows the floor, the problem is in BEAM's emit path at high rates and Phase 4 will show the same.

Acceptance: Compare distribution to Phase 1. If significantly worse, document the delta — that's BEAM's high-emit-rate contribution.

Phase 4 — René vmod emitting same pattern

What: Replace the Tidal pattern with a René voice configured to play the same 16 notes per cycle at the same tempo. Same recording setup as Phase 3.

Why: Adds the René gen_server + per-step pattern evaluation (the 16 notes + 16 skip patterns evaluated each tick). Phase 4 minus Phase 3 isolates vmod-specific overhead.

Andrew's intuition (worth testing explicitly): René in particular may have a worse path than Grids / Repetitor. Possible culprits:

- 32 pattern evaluations per step (16 notes + 16 skip patterns) vs Grids' 7 patterns per step
- `refresh_from_snapshot` calls `set_field` twice per step
- `lists:nth/2` in skip-walking is $O(n)$ per step

Acceptance: Quantify the delta. If René is significantly worse than the dense-Tidal baseline of Phase 3, isolate the per-step overhead.

Phase 4b — Grids and Repetitor at comparable density

What: Repeat Phase 4 with Grids (configured to emit every step, not density-gated) and Repetitor at 16 stepsPerCycle. Same notes, same tempo.

Why: Tests Andrew's hypothesis that René specifically is worse. If Grids/Repetitor are clean and René isn't, the problem is in René's emit path, not the vmod machinery generally.

Acceptance: Direct comparison of Phase 4 (René) vs Phase 4b (Grids, Repetitor) at the same emit rate.

Phase 5 — Polysignal-clocked vmod

What: The Wired session — virtual polyEuclid clocking René. Same recording setup.

Why: Adds the cross-machine bus-read latency. If significantly worse than Phase 4, the polysignal→bus→vmod path is contributing variance — possibly the within-tick ordering issue noted in commit [c891127](#).

Acceptance: Phase 5 minus Phase 4 = cross-machine overhead.

Phase 6 — Modular path

What: Tidal → cv-router → ES-9 → modular VCO/VCA → audio interface input → Ableton recording. Different signal chain entirely (no DAW synth; modular envelope into ADC).

Why: The hardware/CV path has its own variance characteristics. Useful baseline for the live-rig case. Probably the cleanest path musically (analog CV doesn't jitter once it's been DAC'd) but the journey to the DAC matters.

Acceptance: Calibrate cv-router/ES-9 path latency and jitter. Compare to the MIDI path baselines.

Tools

- **MIDI analysis:** Python + `mido` for MIDI file parsing, or our custom parser (already used for the 60/120 BPM jitter analysis on 2026-05-20)
- **Audio onset detection:** `librosa.onset.onset_detect` or `aubio` — onset times to sample accuracy
- **In-Live MIDI timestamping:** Max for Live patch that logs MIDI receive timestamps to a CSV file. TBD whether this exists or we need to write one (~30 lines of M4L).
- **Visualisation:** `matplotlib` histograms + time-series scatter, saved as PNGs alongside the raw data
- **Analysis layer:** a Python module `tools/timing-analysis/analyse.py` that takes (`scheduled_times`, `observed_times`) and produces histogram + summary statistics. Reused across all phases.

Recording approach

Two distinct modes by phase:

- **MIDI-only phases (1, 3):** Skip audio entirely. Log inside Live via the Max for Live patch. No audio interface, no BlackHole, no buffer-stage in the measurement.
- **Audio phases (2, 4, 4b, 5, 6):** Ableton's master record (record audio back into a Live audio track). Set Live's audio buffer to the smallest stable size (probably 128 samples at 96 kHz = ~1.3 ms of buffer-side jitter floor, well under what we're looking for). No BlackHole — it adds its own buffer (~21 ms at default settings) with no benefit for our case.

Reference clock / time-zero

The challenge: aligning Tidal-time (cycle 0 of the abstract clock) with Live's bar-grid (transport beat 0).

Approach: Every test pattern includes a "reference beat" — a single MIDI note fired at Tidal cycle 0 to a dedicated channel (say, ch15) and recorded as a sync click alongside the test pattern. First onset of the click in the recording = $t=0$. All deviations measured relative to that.

The reference beat is part of the test session, not a separate test — it travels with the data.

Analysis output

Per phase, save under `tools/timing-data/phase-N-<description>/:`

- **raw.csv** — scheduled and observed times per event
- **summary.json** — mean offset, stdev, min, max, p50, p95, p99, drift (slope of offset vs time), worst-case sample
- **histogram.png** — distribution of offset values
- **timeseries.png** — offset over time, looking for drift or clustered outliers (GC patterns)
- **README.md** — what was tested, when, what hypothesis it addressed, the conclusion

The directory IS the record. Future investigations should be able to diff a Phase N run today against the same Phase N from earlier to confirm a fix worked.

Hypotheses going in

- **Phase 0** — <1 ms (tool floor)
- **Phase 1** — <5 ms (clean MIDI chain, sparse rate)
- **Phase 2** — Phase 1 + ~5–15 ms constant offset (Ableton buffer)
- **Phase 3** — open. If jitter appears here, problem is BEAM emit path at high rates. If clean, problem is vmod-specific.
- **Phase 4** — open. Most likely shows the 25 ms floor.
- **Phase 4b** — open. If Grids/Repetitor are clean and René isn't, Andrew's intuition is right and the fix is René-local.
- **Phase 5** — Phase 4 + small overhead. Bus reads are O(1). If significantly worse, the within-tick ordering is real.
- **Phase 6** — TBD. Likely cleaner than MIDI because cv-router's scheduled-send is tight.

What we DON'T do

- **No patches.** We measure, document, then design fixes based on data. No **lookAheadMs** bumps, no buffer-size tuning, no optimisation guesses.
- **No code changes mid-investigation.** Code stays stable across phases so the deltas are pure. Fixes come after the picture is complete.
- **No skipping phases** even if we think we know the answer. The baseline data has standalone value.

Order of operations

1. Phase 0 — confirm the tools.
2. Phase 1 → 2 → 3 in order — Tidal pattern complexity increasing.
3. Phase 4 / 4b — vmods.
4. Phase 5 — cross-machine.
5. Phase 6 — modular.

If a phase reveals the dominant bottleneck, we can pause and design the fix before continuing. But ideally we map all of it first so we know what the secondary contributors look like — fixing the dominant one might unmask a smaller one we'd otherwise miss.

Acceptance for the session

The session is "done" when:

- Every phase has a saved data directory under **tools/timing-data/phase-N-.../**

- A summary doc captures: dominant source of variance, its magnitude, designed fix, expected post-fix numbers
- The next steps are concrete: implement the fix, re-run the relevant phase, verify the prediction

We may take more than one calendar session — that's expected. The phases are decoupled enough to pause between them.